



***COMEÇAREMOS
EM BREVE!***





especialização

RAN Intelligent Controller (RIC)

Prof. Dr. Jonas Augusto Kunzler

Open RAN:

Redes Abertas e Tecnologias Emergentes

 Bloco 1 — 18:30–20:00

Pilha integrada OAI + OSC · verificação de saúde da rede

Disciplina 7: RAN Intelligent Controller (RIC)

Aula 05: Design integrado, SMO e validação da pilha OAI+OSC

Data: 18/06/2026 | Horário: 18:30–22:00 | Duração: 3h30

NGO · Polese SMO/O1/R1 · pilha `oai-cn-gnb-nonrt-nearrt`

Non-RT + Near-RT OSC · verificação Core/RAN · Projeto 2





Calendário — destaque Aula 05

Data / Horário	Conteúdos
09/06 18:30–22:00	Aula 01: Retomada O-RAN + Lab Open5GS/UE RAN SIM
11/06 18:30–22:00	Aula 02: Artigo NGO — OSS RIC e casos de uso
13/06 08:00–13:00	Aula 03: Interface A1, políticas e seminários
16/06 18:30–22:00	Aula 04: xApps/rApps · OAI+FlexRIC (E2)
18/06 18:30–22:00	Aula 05: Design integrado · pilha OAI+OSC · SMO
20/06 08:00–13:00	Aula 06: Testes, depuração e seminários

Acordos Didáticos

- Presença: início, **dois intervalos de 10 min** e encerramento.
- Exercícios: resolver os exercícios propostos na aula.
- Planejamento: plano de testes do Projeto 2 em grupo.
- Registrem evidências de saúde da rede (logs, IPs, ping) para o relatório.

Cronograma Previsto — Aula 05

- 18:30–20:00 — Pilha integrada OAI+OSC · verificação Core/RAN · achados do lab
- 20:00–20:10 — Intervalo
- 20:10–20:50 — Fase 2 O-RAN SC (demo KPM) · SMO · design xApp/rApp
- 20:50–21:00 — Intervalo
- 21:00–22:00 — Segurança · observabilidade · Planejamento Projeto 2

Objetivos da Aula 05

Objetivo central

Validar a pilha OAI integrada à OSC (Non-RT + Near-RT) e mapear ao design NGO + SMO

Ao final da aula

Documentar saúde Core/RAN, entender fases do lab e rascunhar plano de testes do Projeto 2

Retomada — Aula 04 → Aula 05

- **Aula 04:** OAI + FlexRIC (Fase 1) — E2 SETUP, KPM, RC, ciclo de vida xApp.
- **Aula 05:** ampliamos para **pilha integrada** — Non-RT OSC + Near-RT OSC + SMO.
- Projeto: `code/oai-cn-gnb-nonrt-nearrt/`
- Doc: `docs/FASES_ORAN_LAB.md` · índice: `docs/E2_FLEXRIC.md`

Fio condutor — NGO (2024) + Polese (2023)

- **NGO Fig. 6:** SMO + Non-RT (OSC) + Near-RT + RAN — campus SUTD.
- **NGO §4–5:** rApps ML + xApps KPM/TS — design ponta a ponta.
- **NGO §6:** validação — métricas, evidências, limitações do testbed.
- **Polese:** SMO, O1, O2, R1, security — governança e assurance.

Projeto `oai-cn-gnb-nonrt-nearrt`

- Core OAI — 5GC Docker (`oai-cn5g-fed` , UPF-VPP).
- RAN — gNB OAI (RFSIM) ou UERANSIM containerizado.
- Fase 1: nonRT OSC + FlexRIC nearRT (E2 :36421).
- Fase 2: nonRT + nearRT O-RAN SC (E2 :36422, A1 real).
- Fase 3: SMO/OAM scaffold (O1 simulado, topologia).

Três fases do lab — visão geral

Fase	nearRT	nonRT	SMO	Porta E2
1	FlexRIC	PMS + A1 sim	—	36421
2	O-RAN SC	PMS → A1 Mediator	—	36422
3	Opcional	Opcional	OAM + O1 sim	—

Regra: Fase 1 e 2 não rodam nearRT simultaneamente.

Discussão 1 — Por que uma pilha integrada?

1. O que o lab da Aula 04 (só FlexRIC) não cobre que a OSC cobre?
2. Qual papel do **Non-RT** se o xApp já roda no Near-RT?
3. Em produção, quem “manda” — SMO, Non-RT ou operador humano?
4. Que parte desta pilha mais chama a atenção do grupo?

Discussão 1 — Por que uma pilha integrada?

1. O que o lab da Aula 04 (só FlexRIC) não cobre que a OSC cobre?

Resposta: O lab da Aula 04 (só FlexRIC) não cobre o papel do Non-RT.

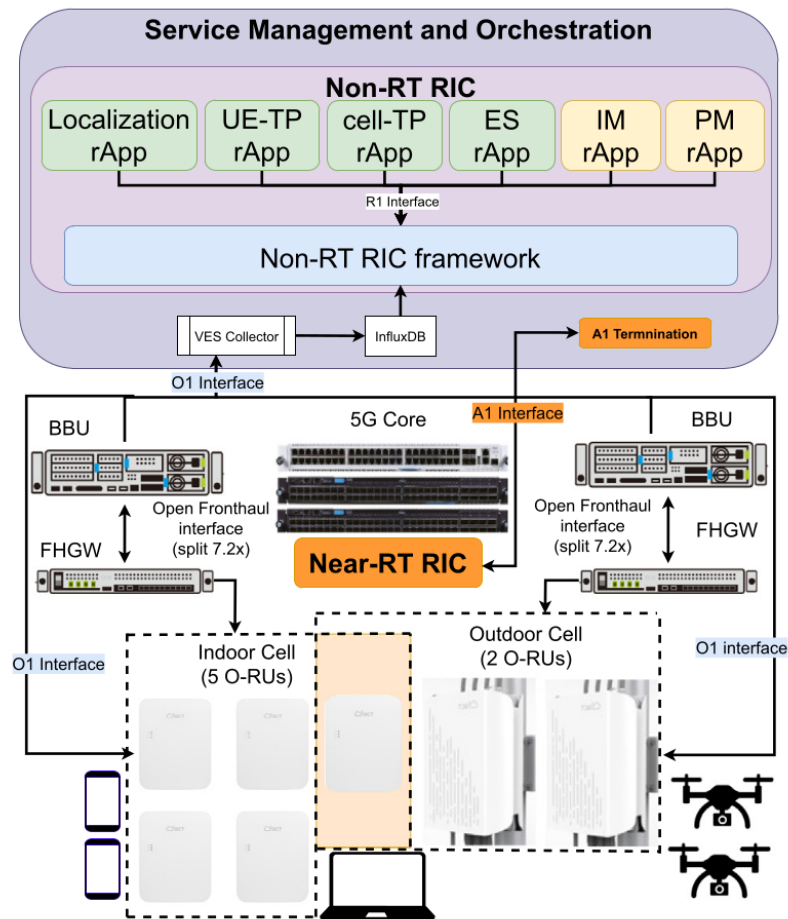
2. Qual papel do **Non-RT** se o xApp já roda no Near-RT?

Resposta: Responsável por gerenciar as políticas e as relações.

3. Em produção, quem “manda” — SMO, Non-RT ou operador humano?

Resposta: Operador humano é o responsável pelas Intenções (Intent).

NGO Fig. 6 adaptado ao nosso lab



- **5G Core + RAN** — OAI
- **Near-RT RIC** — FlexRIC ou O-RAN SC.
- **Non-RT RIC** — OSC PMS, A1 sim (F1) ou A1 Mediator (F2).
- **SMO** — Fase 3.

Contraste SUTD: RAN comercial + proxy E2.
No nosso lab: RAN OAI + RFSIM acadêmico.

Discussão 2 — O2 no lab vs Polese

- **O2 (Polese):** deploy de VNFs/CNFs no SMO — lifecycle do core/RAN.
- **Nosso lab:** Core OAI sobe via `core-network.py` + Docker Compose.
- O `up_core.sh` é um “O2 simplificado” ou apenas script de deploy?
- Que evidência de **saúde** um operador exigiria antes de subir o RIC?

Discussão 2 — O2 no lab vs Polese

- O `up_core.sh` é um “O2 simplificado” ou apenas script de deploy?

Resposta: É um **script de deploy** que chama `core-network.py` — este instancia o 5GC e valida readiness (containers healthy, NRF, N4).

Didaticamente aproxima-se de um **O2 simplificado** (só instantiate + health-check), mas *não* é O2 real: sem SMO, catálogo nem lifecycle (scale/heal/upgrade).

Discussão 2 — O2 no lab vs Polese

- Que evidência de **saúde** um operador exigiria antes de subir o RIC?

Resposta: Em camadas:

- (1) Core — NFs healthy, AMF/SMF/UPF no NRF, N4 SMF↔UPF;
- (2) rede — forwarding, `demo-oai` com .129;
- (3) RAN — PDU session + ping 8.8.8.8 pelo UE;
- (4) só então RIC — containers nearRT/nonRT, E2 SETUP e nó no RNIB
(`test_oran_ric.sh`).

Princípio desta aula — saúde antes do RIC

Antes de Near-RT, Non-RT ou xApps, validamos o **plano de dados 5G**:

1. **Core** — NFs registradas no NRF; SMF↔UPF (N4) ativo.
2. **RAN** — attach UE, PDU session, ping externo (N6).
3. **Interfaces** — IPs corretos em `demo-oai` e túnel UE.

Sem isso, falhas no E2/A1 podem ser sintoma de problema no 5GC, não no RIC.

Procedimento — pré-requisitos

- Docker ativo · ~8 GB RAM livre · Ubuntu 22.04+
- IPv4 forwarding: `net.ipv4.ip_forward=1`
- Diretório: `cd code/oai-cn-gnb-nonrt-nearrt`
- Build gNB (se usar OAI nativo): `build_e2.sh` / `build_flexric_tools.sh`

Passo 1 — Subir o Core OAI

```
ric/code/oai-cn-gnb-nonrt-nearrt$ ./scripts/up_core.sh
```

```
=====
```

```
Iniciando Core OAI (5G CN)
```

```
=====
```

```
Habilitando IP forwarding no host...
```

```
net.ipv4.ip_forward = 1
```

```
net.ipv6.conf.all.forwarding = 1
```

```
Iniciando Core OAI (AMF, SMF, NRF, UPF-VPP, UDM, UDR, AUSF, MySQL, DN)...
```

- Habilita IP forwarding no host
- `core-network.py --type start-basic-vpp --scenario 1`
- Stack **basic-vpp** — UPF-VPP (FD.io), não SPGWU-Tiny
- Health check automático (~30 s após start)

Passo 1 — O que o script verifica

1. Containers **healthy** (Docker healthcheck).
2. AMF, SMF, UPF registrados no **NRF** (192.168.70.130).
3. AUSF, UDM, UDR registrados no NRF.
4. SMF conectado ao UPF — **N4 Association** + heartbeats.

Achado — redes Docker criadas

- `demo-oai-public-net` — Core ↔ RAN (N2/N3)
- `oai-public-core` — NFs internas do 5GC
- `oai-public-access` — GTP-U (gNB ↔ UPF-VPP)
- Interface host `demo-oai` criada pelo Docker

Achado — containers healthy

Container	Imagem	Status
mysql	mysql:8.0	healthy
oai-nrf	oai-nrf:v1.5.1	healthy
oai-amf	oai-amf:v1.5.1	healthy
oai-smf	oai-smf:v1.5.1	healthy
vpp-upf	oai-upf-vpp:v1.5.1	healthy
oai-ausf/udm/udr	v1.5.1	healthy
oai-ext-dn	trf-gen-cn5g	healthy

Achado — registro no NRF (NFs de sessão)

NF	IP registrado	Status
Access and Mobility Management Function (AMF)	192.168.70.132	Accepted
Session Management Function (SMF)	192.168.70.133	Accepted
User Plane Function (UPF)	192.168.70.201	Accepted

NRF: 192.168.70.130 — consulta via `nnrf-nfm/v1/nf-instances`

Achado — registro no NRF (NFs de assinatura)

NF	IP registrado	Status
Authentication Server Function (AUSF)	192.168.70.138	Accepted
Unified Data Management (UDM)	192.168.70.137	Accepted
Unified Data Repository (UDR)	192.168.70.136	Accepted

Mensagem do script: *"OAI 5G Core network is configured and healthy"*

Achado — plano de usuário (N4 / PFCP)

- UPF respondeu ao N4 Association Request do SMF.
- SMF recebe heartbeats do UPF — túnel de sessão operacional.
- UPF-VPP: processamento vetorial GTP-U (N3/N6) — throughput superior ao Tiny.
- Sem N4 ativo, UE pode attachar mas **não navega** (sem PDU session válida).

Discussão 3 — Interpretando “healthy”

1. Docker `healthy` garante que o 5GC **funciona** end-to-end?
2. Por que checar NRF **além** do status do container?
3. Que NF falhando causaria attach sem internet?
4. Como documentar isto no plano de testes?

Discussão 3 — Interpretando “healthy”

1. Docker `healthy` garante que o 5GC funciona end-to-end?

Resposta: Não. O healthy do container não garante que o 5GC funcione end-to-end. É necessário checar o status de todos os NFs registrados.

2. Por que checar NRF **além** do status do container?

Resposta: Para verificar se os NFs estão registrados no NRF.



Discussão 3 — Interpretando “healthy”

3. Que NF falhando causaria attach sem internet?

Resposta: Se a AMF falhar, o UE não conseguirá registrar e attachar. Se a SMF falhar, o UE não conseguirá estabelecer uma sessão PDU. Se a UPF falhar, o UE não conseguirá enviar ou receber dados.

4. Como documentar isto no plano de testes?

Resposta: No plano de testes, deve-se documentar que o 5GC deve estar healthy e que todos os NFs devem estar registrados no NRF.

Achado — IP forwarding no host

```
net.ipv4.ip_forward = 1  
net.ipv6.conf.all.forwarding = 1
```

- Necessário para roteamento UE → DN externa (8.8.8.8)
- `iptables -P FORWARD ACCEPT` aplicado pelo script
- Sem forwarding: ping do UE falha mesmo com Core healthy

Passo 2 — Subir UERANSIM (RAN containerizado)

```
ric/code/oai-cn-gnb-nonrt-nearrt$ ./scripts/up_ueransim.sh
```

- gNB + UE em um container Docker (`ueransim`)
- Rede: `demo-oai-public-net` + `oai-public-access`
- Alternativa rápida para validar N2/N3/N6 sem build OAI
- Health: `Up ... (health: starting)` → estabiliza em ~10 s

Teste — conectividade UERANSIM

```
ric/code/oai-cn-gnb-nonrt-nearrt$ docker exec ueransim ping -c 3 -I uesimtun0
```

Métrica	Valor observado
IP UE	12.1.1.2 (uesimtun0)
Perda	0% (3/3)
RTT médio	~26,5 ms



Discussão 4 — UERANSIM no pipeline

- UERANSIM expõe agente E2 para testes RIC?
- Onde o UERANSIM é relevante para o Projeto 2?

Discussão 4 — UERANSIM no pipeline

- UERANSIM expõe agente E2 para testes RIC?

Resposta: Não. O UERANSIM não expõe agente E2 para testes RIC.
Use o gNB OAI.

- Onde o UERANSIM é relevante para o Projeto 2?

Resposta: Para validar o Core OAI.

Passo 3 — Subir gNB OAI (RFSIM nativo)

```
ric/code/oai-cn-gnb-nonrt-nearrt$ ./scripts/up_gnb_oai.sh
```

- Binários: `nr-softmodem` + `nr-uesoftmodem`
- Configura `192.168.70.129/24` em `demo-oai`
- E2 agent integrado (FlexRIC :36421 ou ORAN-SC :36422 conforme build)
- Logs: `logs/gnb_oai.log` , `logs/ue_oai.log`

Achado — PIDs e logs do gNB OAI

Processo	Exemplo
gNB PID	69032
nrUE PID	69646
Log gNB	logs/gnb_oai.log
Log UE	logs/ue_oai.log

Parar: `./scripts/down_gnb_oai.sh`

Achado — interfaces de rede no host

Interface	IP / papel
demo-oai	192.168.70.1 + 192.168.70.129/24
cn5g-core	192.168.73.1/24
cn5g-access	192.168.72.1/24
oaitun_ue1	12.1.1.3/24 (túnel PDU do nrUE)

💬 Discussão 5 — Dois IPs em demo-oai

- 192.168.70.1 — gateway Docker da rede demo-oai-public-net
- 192.168.70.129 — IP do host para NGAP (gNB → AMF)
- O que quebra se este IP estiver ausente ou duplicado?

Resposta: Se o IP estiver ausente, o gNB não conseguirá se conectar ao AMF. Se o IP estiver duplicado, haverá conflito de registro de gNB no NRF.

Comparativo — UERANSIM vs gNB OAI

	UERANSIM	gNB OAI (RFSIM)
Deploy	Docker	Binários nativos
IP UE	12.1.1.2	12.1.1.3
E2 agent	Não	Sim
Uso no curso	Smoke test Core	Lab RIC + Projeto 2
RTT ~27 ms	✓	✓

Síntese Bloco 1 — checklist de saúde

- ☒ Core OAI healthy + NRF + N4
- ☒ Ping externo via UERANSIM (12.1.1.2)
- ☒ Ping externo via nrUE OAI (12.1.1.3)
- ☐ Near-RT RIC (próximo: Fases 1–2)
- ☐ SMO / O1 (Fase 3)

Evidências → anexo do plano de testes e relatório Projeto 2

 **Intervalo — 20:00–20:10**

 Bloco 2 — 20:10–20:50

Non-RT · Near-RT OSC · SMO · design integrado

Fase 1 — nonRT OSC + FlexRIC (E2 :36421)

```
ric/code/oai-cn-gnb-nonrt-nearrt$ ./scripts/up_e2_lab.sh  
ric/code/oai-cn-gnb-nonrt-nearrt$ ./scripts/test_nonrt_ric.sh --seed
```

- nonRT: PMS :8081, Control Panel :8181, A1 **simuladores**
- nearRT: FlexRIC + xApps C (`xapp_kpm_moni`)
- A1 sim $\neq$ FlexRIC real — políticas não chegam ao gNB nesta fase

Fase 1 — componentes nonRT

Container	Função	Porta
nonrt-policy-agent	PMS (A1 policies)	8081
nonrt-control-panel	UI web	8181
a1-sim-OSC / STD	Simula nearRT A1	30001–30005
nonrtric-gateway	API Gateway	9090

Doc: docs/NONRT_RIC.md · docs/FASE1_NONRT_FLEXRIC.md

Discussão 6 — A1 simulado vs A1 real

- Fase 1: rApp cria policy no PMS → vai para **simulador**, não ao gNB.
- Isso ainda ensina o **ciclo de vida** de política A1?

Resposta: Sim. O A1 simulador tem papel relevante para ensinar o ciclo de vida de política A1.

Discussão 6 — A1 simulado vs A1 real

- Qual dos rApps apresentados no artigo, na Seção 6.1, faria sentido apenas com A1 real na Fase 2?

Resposta: Os rApps que mais justificam A1 real são os de decisão/controle: **ES-rApp** e **IM-rApp**. O ES-rApp decide ligar/desligar O-RUs ou reduzir potência para economia de energia, e o IM-rApp decide reduzir potência de O-RUs vizinhas para mitigar interferência.

Fase 2 — transição Fase 1 → O-RAN SC

```
# para FlexRIC + nonRT Fase 1
ric/code/oai-cn-gnb-nonrt-nearrt$ ./scripts/down_e2_lab.sh
# para Core + nearRT OSC + nonRT A1 + gNB
ric/code/oai-cn-gnb-nonrt-nearrt$ ./scripts/up_oai_oran_lab.sh
```

- FlexRIC **desativado** — porta E2 muda de **36421** para **36422**
- Core OAI pode permanecer em execução (reutilizado)
- Doc: docs/FASE2_ORAN_SC_A1.md

Passo a passo — `up_oai_oran_lab.sh` (4 etapas)

1. [1/4] Core OAI — reutiliza stack healthy (AMF/SMF/UPF-VPP)
2. [2/4] nearRT O-RAN SC — `vendor/oran-sc-ric` + overlay `config/oran-ric/`
3. [3/4] nonRT perfil oran — PMS → `ric_a1mediator` (A1 real)
4. [4/4] gNB + nrUE — `nr-softmodem-oran-sc` → E2 `10.0.2.10:36422`

Achado — containers nearRT O-RAN SC

Container	Imagem / função	Porta
<code>ric_e2term</code>	ric-plt-e2:6.0.4 — E2AP/SCTP	36422
<code>ric_e2mgr</code>	ric-plt-e2mgr:6.0.4	3800
<code>ric_submgr</code>	ric-plt-submgr:0.10.1	—
<code>ric_a1mediator</code>	ric-plt-a1:3.2.2	10000
<code>ric_dbaas</code>	Redis RNIB (SDL)	6379
<code>python_xapp_runner</code>	xApps Python (build local)	—
<code>ric_rtmgr_sim</code>	Routing Manager Simulator	—

Achado — nonRT perfil Fase 2 (A1 real)

- Rede: `oran-nonrt-oran-net`
- `nonrt-policy-agent-oran` — PMS healthy
- `nonrt-ric-gateway-oran` + `nonrt-control-panel-oran`
- Control Panel: **`http://127.0.0.1:8181/`**
- Sem simuladores A1 — destino é `ric_a1mediator`

Achado — gNB OAI para O-RAN SC

Item	Valor observado
Binário	<code>nr-softmodem-oran-sc</code>
E2 destino	<code>10.0.2.10:36422</code>
Service Models	<code>flexric-lib/</code> (KPM, RC, MAC...)
Log gNB	<code>logs/gnb_oai_oran.log</code>
Log UE	<code>logs/ue_oai_oran.log</code>
UE túnel	<code>oaitun_ue1</code> → 12.1.1.5

Achado — E2 SETUP (Fase 2)

```
[E2 AGENT]: nearRT-RIC IP = 10.0.2.10, PORT = 36422  
[E2-AGENT]: E2 SETUP-REQUEST tx  
[E2-AGENT]: E2 SETUP RESPONSE rx  
[E2-AGENT]: Transaction ID ... 0 ... 0
```

- RAN type: `ngran_gNB` , `nb_id` = 3584
- Plugins carregados: KPM, RC, MAC, RLC, PDCCP, GTP, slice, TC

Achado — RNIB (inventário E2)

- Redis SDL: `ric_dbaas` — 1 entrada RAN
- **E2 Node ID:** `gnb_208_095_000000e00`
- Chave Radio Network Information Base: `{e2Manager}, RAN: gnb_208_095_000000e00`
- `e2mgr e2t list` — associado ao `ric_e2term`
- Comando: `./scripts/get_oran_e2_node_id.sh`

Validação — `test_oran_ric.sh`

Check	Resultado
ric_e2term running	OK
ric_a1mediator running	OK
A1 mediator TCP :10000	OK
PMS (perfil oran)	OK
RNIB RAN entries	OK (1)
E2 SETUP no gNB	OK

Armadilha — `test_oran_ric.sh --run-xapp`

- Timeout 25 s pode encerrar xApp sem INDICATION visível
- Causas: UE ainda attachando; sem tráfego; janela curta
- Log: `logs/test_oran_ric_xapp.log`
- **Solução didática:** `./scripts/run_xapp_oai_kpm_extended.sh`
- Ou: `KPM_TRAFFIC=1` + duração maior

Demo — xApp KPM estendido (O-RAN SC)

```
ric/code/oai-cn-gnb-nonrt-nearrt$ ./scripts/run_xapp_oai_kpm_extended.sh
```

- xApp: `simple_xapp_oai.py` no `python_xapp_runner`
- **KPM Style 4 · S-NSSAI 222/123**
- Subscription ID: `3FJJB0eLpckgNVRlgBCFzNazJHl`
- Métricas: UEThp, PdcpcVolume, RlcDelay, PrbTot
- Doc: `docs/XAPP_KPM_OAI_ORAN.md`

Achado — baseline (sem carga)

Métrica KPM	Valor típico (idle)
DRB.UETHpDL/UL	~1,5–2,2 kbps
DRB.PdcpSduVolumeDL/UL	0 Mb
DRB.RlcSduDelayDL	0–300 μ s
RRU.PrbTotDL	0–1 %
RRU.PrbTotUL	~4 %

INDICATIONs periódicas ~1/s — ciclo **Operate** do xApp ativo

Demo — gerar tráfego + medir throughput

```
# Terminal 1 — xApp KPM (já rodando)
ric/code/oai-cn-gnb-nonrt-nearrt$ ./scripts/run_xapp_oai_kpm_extended.sh
# Terminal 2 — iperf3 via UPF-VPP
$ UE_SOURCE=nrue ./scripts/test-vpp-throughput.sh
```

- UE: `oaitun_ue1` · IP `12.1.1.5`
- Destino DN: `192.168.73.135` (iperf3 server no Core)
- Tráfego uplink pesado → KPM reflete carga em tempo quase real

Achado — KPM sob carga (iperf3)

Métrica	Sem carga	Com iperf3 (~10 s)
DRB.UEThpUL	~2 kbps	~50 000 kbps (≈50 Mbps)
DRB.UEThpDL	~2 kbps	~800–920 kbps
DRB.PdcpSduVolumeUL	0 Mb	70–99 Mb/intervalo
RRU.PrbTotUL	~4 %	99 %
DRB.RlcSduDelayDL	0 μs	~130–200 μs

Evidência — KPM + iperf3 em paralelo

```
DRB.PdcpSduVolumeUL: 92.000 Mb
DRB.RlcSduDelayDL: 151.482 us
DRB.UETHpDL: 1585.088 kbps
DRB.UETHpUL: 93150.872 kbps
RRU.PrbTotDL: 1.000 %
RRU.PrbTotUL: 99.000 %

RIC Indication from gnb_208_095_00000e00 (sub 1)
receivedAt: 2026-06-18 18:48:02 UTC
kpmTimeRaw: 623d346a
kpmTimeDecoded: 2026-06-18 18:48:02 UTC
UE_id: 2
  DRB.PdcpSduVolumeDL: 1.000 Mb
  DRB.PdcpSduVolumeUL: 94.000 Mb
  DRB.RlcSduDelayDL: 126.565 us
  DRB.UETHpDL: 1640.408 kbps
  DRB.UETHpUL: 94805.424 kbps
  RRU.PrbTotDL: 1.000 %
  RRU.PrbTotUL: 99.000 %

RIC Indication from gnb_208_095_00000e00 (sub 1)
receivedAt: 2026-06-18 18:48:03 UTC
kpmTimeRaw: 633d346a
kpmTimeDecoded: 2026-06-18 18:48:03 UTC
UE_id: 2
  DRB.PdcpSduVolumeDL: 1.000 Mb
  DRB.PdcpSduVolumeUL: 98.000 Mb
  DRB.RlcSduDelayDL: 136.161 us
  DRB.UETHpDL: 1703.096 kbps
  DRB.UETHpUL: 98804.024 kbps
  RRU.PrbTotDL: 1.000 %
  RRU.PrbTotUL: 99.000 %

RIC Indication from gnb_208_095_00000e00 (sub 1)
receivedAt: 2026-06-18 18:48:04 UTC
kpmTimeRaw: 643d346a
kpmTimeDecoded: 2026-06-18 18:48:04 UTC
UE_id: 2
  DRB.PdcpSduVolumeDL: 1.000 Mb
  DRB.PdcpSduVolumeUL: 90.000 Mb
  DRB.RlcSduDelayDL: 172.812 us
  DRB.UETHpDL: 1457.136 kbps
  DRB.UETHpUL: 90617.176 kbps
  RRU.PrbTotDL: 1.000 %
  RRU.PrbTotUL: 99.000 %

jakunzler@jakunzler-960XGL: ~/Documents/GitHub/cesar/cesar-school-repo/ric/code/oai-cn-gnb-nonrt-neart
=====
Teste de Throughput VPP (iperf3)
=====

Este teste mede o throughput do plano de usuário 5G
através do UPF-VPP (Vector Packet Processing).

Verificando containers...
✓ Containers OK

Usando nrUE (interface oaitun_ue1)

Iniciando servidor iperf3 no DN (oai-ext-dn)...
Executando teste de throughput (10s, modo tcp)...
-----
Connecting to host 192.168.73.135, port 5201
[ 5] local 12.1.1.3 port 36629 connected to 192.168.73.135 port 5201
[ ID] Interval      Transfer       Bitrate        Retr  Cwnd
[ 5]  0.00-1.00    sec  13.2 MBytes   111 Mbits/sec    0   604 KBytes
[ 5]  1.00-2.00    sec  12.1 MBytes   102 Mbits/sec    0   1.13 MBytes
[ 5]  2.00-3.00    sec  11.0 MBytes   92.3 Mbits/sec   0   1.68 MBytes
[ 5]  3.00-4.00    sec  10.4 MBytes   87.0 Mbits/sec   0   2.22 MBytes
[ 5]  4.00-5.00    sec  10.0 MBytes   83.9 Mbits/sec   0   2.75 MBytes
[ 5]  5.00-6.00    sec  12.0 MBytes   101 Mbits/sec   0   3.31 MBytes
[ 5]  6.00-7.00    sec  10.6 MBytes   89.1 Mbits/sec   0   3.69 MBytes
```

Troubleshooting Fase 2 — lições do lab

Sintoma	Causa / ação
xApp timeout sem INDICATION	Usar <code>run_xapp_oai_kpm_extended.sh</code> + tráfego
Porta E2 em conflito	<code>down_e2_lab.sh</code> antes de Fase 2
gNB aborta no unsubscribe	Ctrl+C sem delete subscription (workaround documentado)
A1 healthcheck 404	Path da API mudou — usar PMS + E2 como evidência principal

FlexRIC vs O-RAN SC — comparativo

	FlexRIC (F1)	O-RAN SC (F2)
Deploy	Binário host	Docker multi-container
Porta E2	36421	36422
xApps	C (exemplos)	Python / xApp frame
A1	Não integrado	A1 Mediator
Didática	E2SM direto	Stack produção OSC

Discussão 7 — Qual nearRT no Projeto 2?

1. FlexRIC é “mais simples” — isso é vantagem ou limitação?
2. O-RAN SC exige mais infra — quando vale o esforço extra?
3. Podem comparar **mesma métrica KPM** nas duas fases?
4. Porta E2 errada — sintoma típico que vocês já viram?

Fase 3 — SMO/OAM (scaffold)

- SMO: Keycloak, VES, SDNC, TEIV (topologia/inventário)
- **O1 simulado** — ntsim-ng (O-DU/O-RU); gNB OAI sem O1 real
- Isolada das Fases 1–2 (portas em conflito)
- `SMO_OAM_DIR` — checkout externo O-RAN SC OAM
- Doc: `docs/FASE3_SMO_OAM.md`

SMO — Polese + NGO (papel no caso SUTD)

- O1 — ES-rApp (sleep RRU); PM-rApp (alarmes/performance).
- O2 — deploy VNFs/CNFs; no lab: `up_core.sh` + Compose.
- R1 — SMO ↔ rApps (intent, dashboards operador).
- Assurance — SLA vs políticas A1 (feedback loop).

Fluxo integrado — SMO → rApp → xApp → RAN

1. Operador define **intent** no SMO (ou Control Panel nonRT).
2. **rApp** deriva política **A1** (Fase 2: via Mediator).
3. **xApp** near-RT subscribe/control via **E2**.
4. Telemetria **O1** + **E2/KPM** sobe para assurance.
5. Lab Fase 1: passos 3–4 sem SMO real; Fase 3 adiciona O1 sim.

Design de xApp — checklist (Near-RT)

- Quais E2SM e report/control styles? (KPM v2.03, RC...)
- Slice lab: **SST=222, SD=123** — filtro KPM alinhado.
- Periodicidade vs carga no nó RAN (RFSIM).
- Falhas: timeout E2, porta errada, E2SM não suportado.
- FlexRIC: `-p flexric-lib/` · OSC: xApp runner + RNIB.

Design de rApp — NGO §4 (ML pipeline)

- **Localization-rApp** — LSTM; **UE-TP-rApp** — regressão.
- Treino offline Non-RT; inferência + política via **A1**.
- Métricas alvo NGO §6: $\text{acc} > 90\%$, $R^2 > 0,85$.
- No lab: seed de policies no Control Panel (`test_nonrt_ric.sh --seed`).
- Telemetria/drift visíveis no SMO (O1 + DMaaP) — Fase 3.

SDK e APIs — duas stacks Near-RT

Stack	SDK / runtime	Teste rápido
FlexRIC	C callbacks E2	<code>test_e2_kpm.sh</code>
O-RAN SC	Python xApp frame	<code>test_oran_ric.sh --run-xapp</code>

Abstrair lógica de negócio do codec ASN.1 — mesmo princípio em ambas.

 **Intervalo — 20:50–21:00**



Bloco 3 — 21:00–22:00

Segurança · observabilidade · Projeto 2

Native AI

- IA distribuída: treino Non-RT, inferência Near-RT.
- Governança de modelos e dados no SMO.
- rApps como containers de lógica AI sobre A1.
- Lab: simular pipeline com policies + KPM, sem ML completo.

Segurança — superfícies na pilha integrada

- **E2/A1** — autenticação, Transmission Layer Security (TLS), Role-Based Access Control (RBAC).
- **xApp malicioso** — sandbox, whitelisting E2SM.
- **Control Panel :8181** — não expor em rede pública.
- **Supply chain** — imagens OAI/OSC versionadas (v1.5.1).

Observabilidade — o que monitorar

- **Core:** `docker ps` , NRF, logs AMF/SMF.
- **RAN:** attach UE, `oaitun_ue1` , ping N6.
- **nearRT:** `logs/nearRT-RIC.log` OU `docker logs ric_e2term` .
- **nonRT:** PMS `/status` , Control Panel policies.
- **E2:** subscription ID, latência indication→control.

Workshop — plano de testes (Projeto 2)

Preencher em grupo:

- CT-01 — `up_core.sh` + evidência NRF/N4
- CT-02 — UERANSIM ou gNB + ping 8.8.8.8
- CT-03 — E2 KPM FlexRIC: `test_e2_kpm.sh` + screenshot

Comandos de referência — pilha completa

```
# Saúde 5G
$ ./scripts/up_core.sh && ./scripts/up_gnb_oai.sh
$ ping -c 3 -I oaitun_ue1 8.8.8.8

# Fase 1 (FlexRIC)
$ ./scripts/up_e2_lab.sh && ./scripts/test_e2_kpm.sh

# Parar
$ ./scripts/down_e2_lab.sh
```

Comandos de referência — pilha completa

```
# Saúde 5G
$ ./scripts/up_core.sh && ./scripts/up_gnb_oai.sh
$ ping -c 3 -I oaitun_ue1 8.8.8.8
```

```
# Fase 2 (O-RAN SC)
$ ./scripts/up_oai_oran_lab.sh
$ ./scripts/test_oran_ric.sh
$ ./scripts/run_xapp_oai_kpm_extended.sh
$ UE_SOURCE=nrue ./scripts/test-vpp-throughput.sh
```

```
# Parar
$ ./scripts/down_oai_oran_lab.sh
```

Material de apoio — Aula 05

- NGO et al. — Fig. 6, §4–6 (design e validação).
- Polese et al. — SMO, O1, O2, R1, security.
- Lab integrado — `code/oai-cn-gnb-nonrt-nearrt/docs/FASES_ORAN_LAB.md`
- Fase 2 O-RAN SC — `code/oai-cn-gnb-nonrt-nearrt/docs/FASE2_ORAN_SC_A1.md`
- xApp KPM OAI — `code/oai-cn-gnb-nonrt-nearrt/docs/XAPP_KPM_OAI_ORAN.md`
- Plano de testes — `docs/labs/04-projeto2-plano-testes.md`
- Roteiro Aulas 04–06 — `docs/labs/02-oai-cn-gnb-flexric-e2.md`

Próxima aula

Aula 06 — NGO §6 validação + seminários Projeto 2

- Seminários 08:00–11:00 — apresentações finais (20 min/grupo)
- Debug E2/A1 · métricas NGO
- Tragam diagrama + logs + plano de testes preenchido



Referências

NGO, M. V. et al. "RAN Intelligent Controller (RIC)..." *ICT Express* 10 (2024). (Fig. 6; §4–6)

Polese, M. et al. "Understanding O-RAN." *IEEE ComST*, 2023. (SMO, O1, O2, R1, security)

SUN, J. et al. *O-RAN Native AI Architecture*. RR-2023-02, 2023.



Referências complementares

O-RAN WG2 — SMO architecture

Lab integrado — `code/oai-cn-gnb-nonrt-nearrt/docs/FASES_ORAN_LAB.md`

Non-RT — `docs/NONRT_RIC.md` · **Fase 2** — `docs/FASE2_ORAN_SC_A1.md`

Plano de testes — `docs/labs/04-projeto2-plano-testes.md`



Jonas Augusto Kunzler



Email: jak@cesar.school



LinkedIn: [jakunzler](#)



GitHub: [jakunzler](#)



Instagram: [@jakunzler](#)